

Documento Base Conceitual de Teste

Projeto: TOT — Pinhalense Auxílio de Pesquisa e Automatização

Tecnologia: Flutter (Dart) + Firebase + Groq API (Llama 4 Scout)

Arquitetura: Stateful Widgets + Services + Backend Firebase

Norma aplicada: ISO/IEC/IEEE 29119-1

1. Sistema sob teste

- Aplicativo TOT — chat assistido por IA para recomendação de layouts a partir de croquis enviados pelo usuário, com gestão de banco de imagens de referência e autenticação Firebase.

2. Itens de teste

- LoginScreen
- RegisterPage
- RecoveryPage
- HomePage (chat)
- AdminBancoPage
- IaService (chamada à API Groq / Llama 4 Scout)
- sanitize() — saneamento de entradas
- Fluxo Login → Home
- Fluxo Cadastro → Login
- Fluxo Envio de croqui → resposta da IA
- Fluxo Adicionar/Remover imagem do banco

3. Escopo

- Autenticação (login, cadastro, recuperação)
- Validação de campos de formulário
- Saneamento de entradas contra XSS / injeção
- Navegação entre telas
- Envio de imagem ao Firebase Storage
- Chamada ao IaService e tratamento da resposta
- Gestão do banco de imagens de referência
- Testes de unidade
- Testes de integração

4. Fora de escopo

- Chamadas reais à API da Groq (mockadas)
- Chamadas reais ao Firebase (mockadas via FakeFirebase)
- Testes de performance e carga
- Testes de segurança avançada (pen test)
- Qualidade visual do modelo Llama 4 Scout
- Importação/exportação Excel (Épico 1 — não escopo desta fase)

5. Requisitos

- RF01 — O sistema deve receber um croqui e recomendar o layout mais próximo do banco.
- RF02 — O sistema deve impedir cadastro com campos vazios.
- RF03 — O sistema deve impedir cadastro com e-mail inválido.
- RF04 — O sistema deve permitir o login do usuário.
- RF05 — O sistema deve impedir login com campos vazios.
- RF06 — O sistema deve impedir login com credenciais inválidas.
- RF07 — O sistema deve sanear entradas de texto contra XSS.
- RF08 — O sistema deve permitir adicionar imagens ao banco.
- RF09 — O sistema deve permitir remover imagens do banco.
- RF10 — O laService deve retornar erro quando GROQ_API_KEY não estiver configurada.
- RF11 — O laService deve retornar erro quando o banco de imagens estiver vazio.
- RF12 — O laService deve limitar o payload a 3 imagens de referência.
- RF13 — O sistema deve exibir a imagem recomendada e a mensagem da IA no chat.
- RF14 — O sistema deve tratar índice 0 (sem correspondência) sem quebrar.

6. Condições de teste

- CT01 — Validar saneamento de entrada com tag HTML
- CT02 — Validar saneamento de entrada com caracteres especiais
- CT03 — Validar saneamento de entrada vazia
- CT04 — Validar login com campos vazios
- CT05 — Validar atalho de login admin (admin/123)
- CT06 — Validar login com credenciais inválidas
- CT07 — Validar cadastro com dados válidos
- CT08 — Validar cadastro com campos vazios
- CT09 — Validar cadastro com e-mail inválido
- CT10 — Validar adição de imagem ao banco
- CT11 — Validar remoção de imagem do banco
- CT12 — Validar laService sem GROQ_API_KEY
- CT13 — Validar laService com banco de imagens vazio
- CT14 — Validar laService com retorno válido (índice 1..N)
- CT15 — Validar laService com retorno índice 0 (sem correspondência)
- CT16 — Validar limite de 3 imagens enviadas ao modelo
- CT17 — Validar envio de mensagem só de texto (sem imagem)

7. Tipos de teste

- Teste de Unidade
- Teste de Integração

8. Riscos

- R01 — Chave GROQ_API_KEY exposta em build público
- R02 — laService aceitar índice fora do intervalo válido
- R03 — Saneamento não remover todas as tags maliciosas
- R04 — Login admin (admin/123) permitir acesso em produção
- R05 — Banco de imagens vazio travar a UI

- R06 — Resposta da Groq fora do formato JSON esperado
- R07 — Timeout da API não tratado, deixando a UI suspensa
- R08 — Imagem do banco com storagePath ausente derrubar o fluxo
- R09 — Mensagens de erro técnicas vazarem para o usuário final
- R10 — Estado de upload (`_isUploading`) ficar inconsistente após erro